

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Proyecto Final

Materia: Análisis De Algoritmos

Sección: D01

Profesor: Jorge Ernesto López Arce Delgado

Equipo:

- **Nombre:** IA
- **Integrantes:**
 - Ismael Gándara Cornejo
 - Angel Miguel Villalvazo Vazquez

Carrera: Ingeniería en Computación

Fecha: 26 de noviembre del 2025

Introducción

El problema del Ancestro Común Más Bajo (LCA) es fundamental en el campo de la teoría de grafos para determinar la jerarquía y relación entre los nodos dentro de la estructura de un árbol. Este problema se puede aplicar en la práctica en sistemas de control de versiones (git), bioinformática (árboles filogenéticos) y redes de computadoras.

El siguiente proyecto consiste en el desarrollo de un programa en Python que permite calcular el LCA mediante tres técnicas diferentes (Fuerza bruta, Binary Lifting (divide y vencerás y programación dinámica) y Tarjan), además de poder visualizar paso a paso su ejecución. También se integra una función para la compresión de datos basada en Huffman aplicando así, los algoritmos voraces.

Objetivo

Objetivo General

Desarrollar una aplicación con interfaz gráfica que permita simular, visualizar y comparar el rendimiento de diferentes algoritmos para la búsqueda del LCA, implementando Huffman para la persistencia de datos comprimidos.

Objetivos Específicos

1. Implementar y comparar tres estrategias de búsqueda de LCA: Fuerza bruta, Binary Lifting y Tarjan offline.
2. Desarrollar una visualización gráfica de los recorridos y los saltos realizados por los algoritmos en el árbol.
3. Medir y graficar la complejidad temporal y espacial de cada técnica.
4. Implementar el algoritmo de Huffman para serializar y guardar la estructura del árbol en archivos binarios comprimidos (.huff).

Desarrollo

Descripción del Problema

Dado un árbol y dos nodos u y v , el objetivo es encontrar el nodo más profundo que sea ancestro de ambos.

- **Reto de Eficiencia:** En árboles muy grandes, recorrer desde los nodos hasta la raíz repetidamente es ineficiente. Se requieren métodos de preprocesamiento.
- **Reto de Almacenamiento:** Guardar estructuras de grafos complejas en texto plano (JSON) puede ser costoso en espacio; se requiere compresión.

Implementación de Técnicas

Algoritmo de Compresión de Huffman

Es un algoritmo que asigna códigos binarios de diferente tamaño a los caracteres, basándose en la frecuencia de aparición. Los caracteres más frecuentes obtienen códigos

más cortos. En este proyecto lo utilizamos para comprimir el JSON que define la estructura del árbol. Implementación en código:

El código utiliza un heap con la librería `heapq` para construir el árbol de frecuencias.

Python

```
def huffman_encode_bytes(self, s: str):
    # s es la string pa
    frecuencias = self._calcular_frecuencias(s)
    raiz = self._construir_arbol_huffman(frecuencias)
    codigos = self.generar_codigos(raiz)
    bits = self.codificar_texto(s, codigos)
    padding = (8 - len(bits) % 8) % 8
    bits_padded = bits + "0" * padding

    b_array = bytearray()
    for i in range(0, len(bits_padded), 8):
        byte_str = bits_padded[i:i + 8]
        b_array.append(int(byte_str, 2))

    cabecera = {
        'frecuencias': dict(frecuencias),
        'padding': padding
    }
    cabecera_json = json.dumps(cabecera)
    return cabecera_json.encode('utf-8'), bytes(b_array)
```

Evidencia:

Se genera un archivo `.huff` para guardar el árbol para después poder cargarlo. Al guardar, el sistema muestra la tasa de compresión.

Visualizador de LCA - Completo

1. Simular Preprocesamiento

Simular DFS (Niveles y Padres)

Simular Tabla de Saltos

2. Medir y Simular Búsqueda

N1: 7 N2: 4

☒ Divide y Vencerás

☐ Fuerza Bruta

☐ Tarjan Offline (DSU)

Calcular, Medir y Simular

Resultado: ...

3. Análisis de Complejidad

Ver Gráficos de Complejidad

Limpiar Datos

4. Guardar/Cargar Estructura (Comprimida)

Guardar Estructura Comprimida (.huff)

Cargar Estructura Comprimida (.huff)

Éxito

Estructura comprimida guardada:
C:/Users/PC/Analisis-de-Algoritmos/Practicas/arbol para
reporte.huff

Tamaño JSON: 484 bytes
Tamaño .huff: 447 bytes
Tasa de compresión: 7.64%

Aceptar

LCA con Fuerza Bruta

Primero se igualan los niveles de ambos nodos moviendo el más profundo hacia arriba. Luego, ambos nodos suben simultáneamente nivel por nivel hasta encontrarse.

Implementación en código:

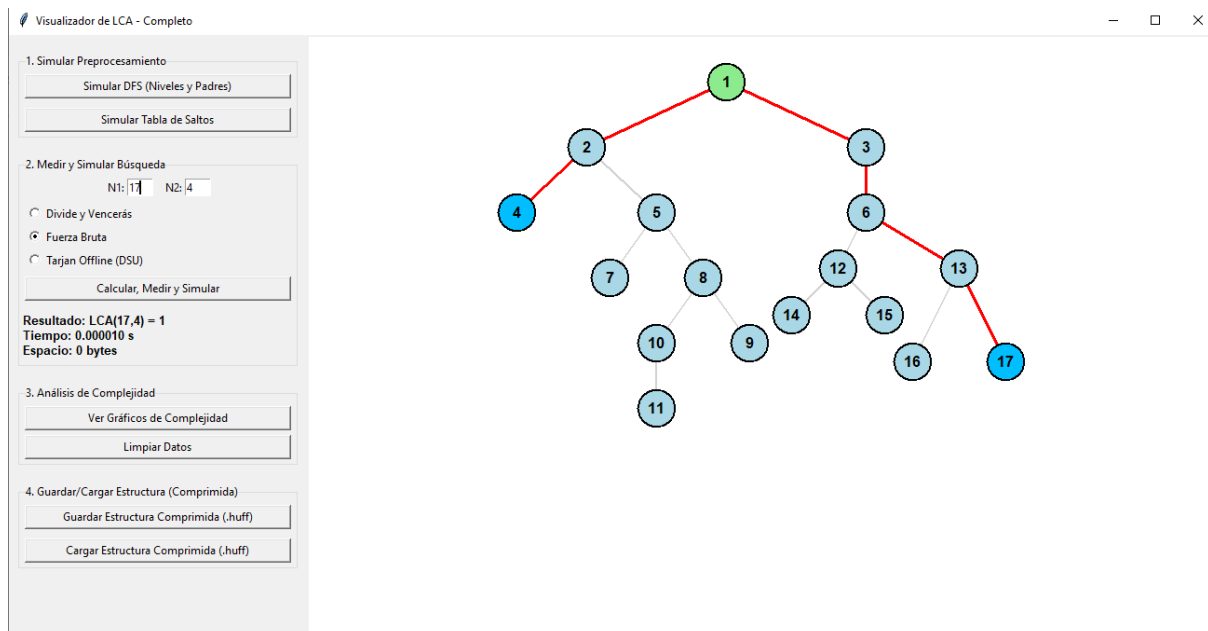
Tiene una complejidad de búsqueda de $O(N)$ en el peor caso.

Python

```
def obtener_lca_fuerza_bruta(self, u, v):  
    while self.nivel[u] > self.nivel[v]:  
        u = self.padre[u][0]  
    while self.nivel[v] > self.nivel[u]:  
        v = self.padre[v][0]  
    while u != v:  
        u = self.padre[u][0]  
        v = self.padre[v][0]  
    return u
```

Evidencia:

Cuando se presiona “Calcular, Medir, Simular” con la opción de fuerza bruta, se muestra el recorrido que hace desde los nodos hasta el LCA.



LCA con Binary Lifting, divide y vencerás con programación dinámica

Se precalcula una tabla $\text{padre}[u][i]$ que almacena el ancestro de u a una distancia de 2^i . Esto permite dar "saltos" grandes hacia la raíz.

Implementación en código:

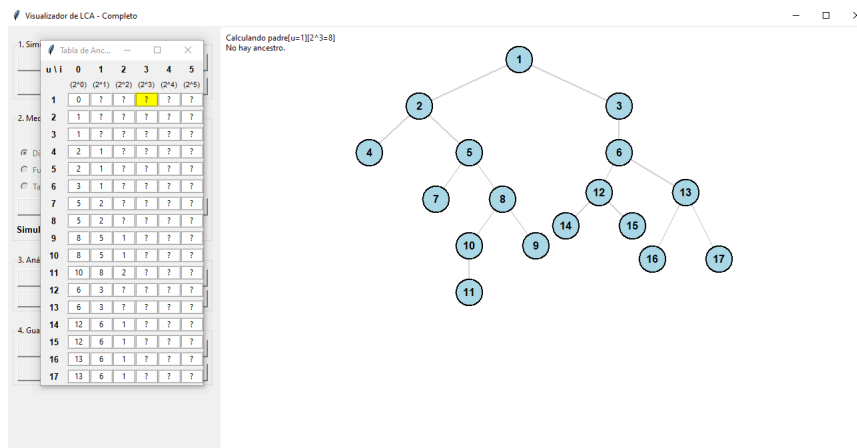
- **Preprocesamiento:** $O(N \log N)$.
- **Consulta:** $O(\log N)$.

Python

```
# Preprocesamiento en función aparte
def obtener_lca(self, u, v):
    if self.nivel[u] > self.nivel[v]:
        u, v = v, u
    for i in range(self.log_max, -1, -1):
        if self.nivel[v] - (1 << i) >= self.nivel[u]:
            v = self.padre[v][i]
    if u == v:
        return u
    for i in range(self.log_max, -1, -1):
        if self.padre[u][i] != 0 and self.padre[u][i] != self.padre[v][i]:
            u = self.padre[u][i]
            v = self.padre[v][i]
    return self.padre[u][0]
```

Evidencia:

En la foto se muestra la simulación de la generación de la tabla de saltos.



Algoritmo de Tarjan

Utiliza la estructura de datos Union-Find (DSU - Disjoint Set Union). Recorre el árbol en DFS y responde a las consultas (queries) cuando visita los nodos. Es "Offline" porque requiere conocer las parejas de nodos (u, v) antes de iniciar el recorrido.

Implementación en código:

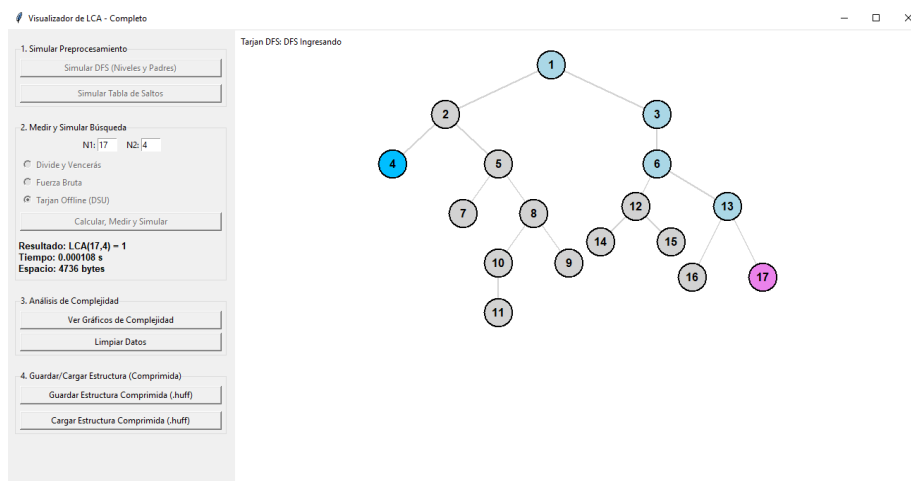
Complejidad casi lineal $O(N + Q\alpha(N))$, donde α es la inversa de la función de Ackermann.

Python

```
def tarjan_dfs(self, u):
    self.dsu_set(u)
    self.dsu_ancestro[u] = u
    for v in self.lista_adyacencia[u]:
        if self.color[v] == "WHITE": # Si es hijo y no ha sido visitado
            if self.padre[v][0] == u:
                self.tarjan_dfs(v)
                self.dsu_union(u, v)
                self.dsu_ancestro[self.dsu_encontrar(u)] = u
    self.color[u] = "BLACK"
    # Verificar consultas u
    for v in self.queries[u]:
        if self.color[v] == "BLACK":
            # LCA es ancestro del conjunto donde v
            self.resultado_tarjan = self.dsu_ancestro[self.dsu_encontrar(v)]
```

Evidencia:

Cuando se presiona “Calcular, Medir, Simular” con la opción de Tarjan Offline, se puede ver el recorrido que se hace desde la raíz con un DFS, cuando llega a uno de los nodos a buscar lo marca y sigue, al llegar al final se regresa a la bifurcación inmediata y sigue con el DFS.



Comparación de Desempeño y Resultados

Utilizamos las librerías matplotlib y tracemalloc para generar gráficas en tiempo real.

Resultados Observados:

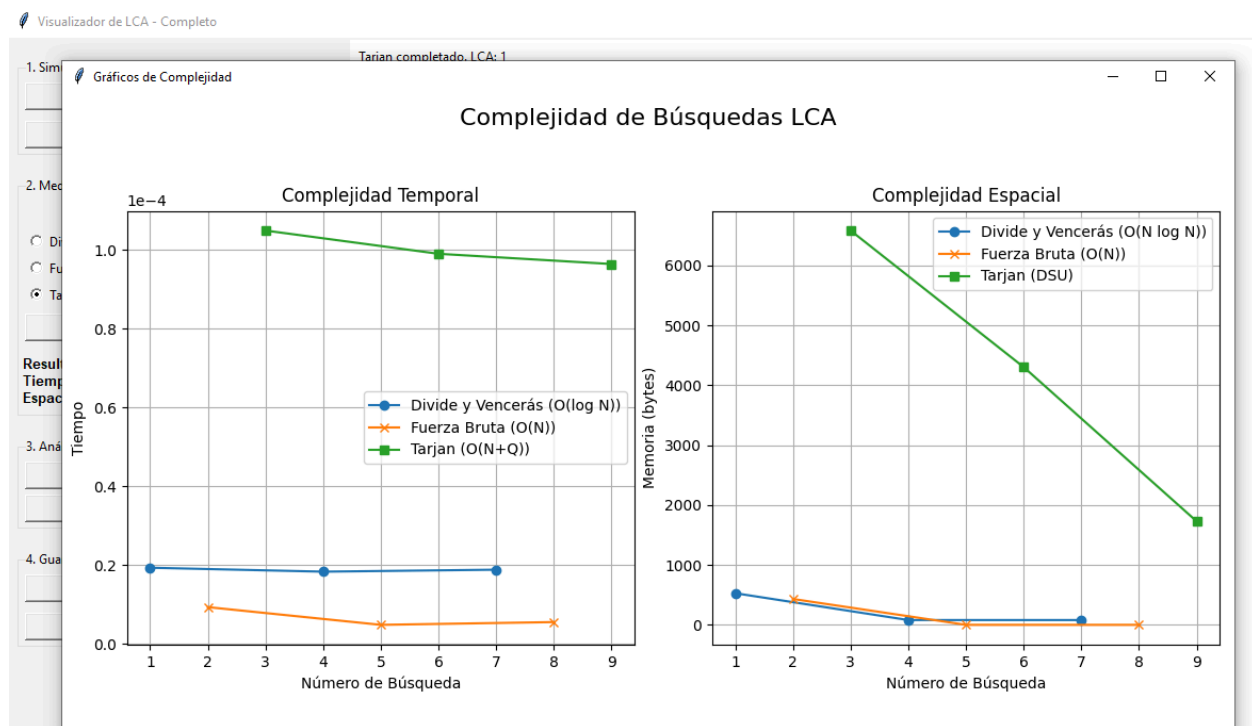
1. Tiempo de Ejecución:

- **Binary Lifting:** Extremadamente rápido para las consultas $O(\log N)$, pero requiere un tiempo considerable de preprocesamiento.
- **Fuerza Bruta:** Su tiempo crece linealmente con la profundidad de los nodos. Es más lento en rutas largas.
- **Tarjan:** Muy eficiente, resolviendo la consulta durante un único recorrido DFS completo, lo que hace que las queries siguientes sean en tiempo casi lineal.

2. Consumo de Memoria:

- **Binary Lifting:** Consume más memoria debido a la tabla de saltos de tamaño $N * \log N$.
- **Fuerza Bruta:** Consume menos memoria (solo necesita punteros al padre, prácticamente nula).
- **Tarjan:** Requiere estructuras auxiliares para el DSU (padre, rango, ancestro).

Diagramas Generados por el Sistema:



En las gráficas podemos observar la eficiencia de estos algoritmos, debido a la falta de nodos, se puede llegar a deducir que el algoritmo de Fuerza Bruta es el mejor de todos, sin embargo, como se explicará más adelante, cada uno tiene sus debilidades y fortalezas dependiendo del contexto del problema y de la entrada.

Conclusión

El desarrollo de los programas nos ha permitido ver que no existe un algoritmo perfecto para cualquier caso, sino que depende de las restricciones y el contexto del problema:

1. Si el árbol es estático y se realizarán muchas de consultas, Binary Lifting es la mejor opción por su velocidad de consulta logarítmica, a pesar de usar más memoria.
2. Si el espacio de memoria es importante y el árbol no es profundo, la Fuerza Bruta es viable por su implementación simple.
3. Si se conocen todas las consultas con anterioridad, Tarjan ofrece un rendimiento casi lineal.
4. El añadir Huffman nos muestra que también se pueden comprimir estructuras como grafos.

En general, la interfaz gráfica nos facilita la comprensión de conceptos poco tradicionales como el binary lifting y la unión de los conjuntos en Tarjan.

Referencias

- Aprende Programación Competitiva. (2022, octubre 24). Olimpiada-informatica.org. <https://aprende.olimpiada-informatica.org/algoritmia-lca>
- Codificación Huffman. (2024, junio 12). StudySmarter ES. <https://www.studysmarter.es/resumenes/ciencias-de-la-computacion/representacion-de-datos-en-ciencias-de-la-computacion/codificacion-huffman/>
- Tarjan's off-line lowest common ancestors algorithm. (2016, mayo 18). GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/tarjans-off-line-lowest-common-ancestors-algorithm/>